Rebuilding Computational Physics in Rust with AI Agents

Hiroshi Shinaoka

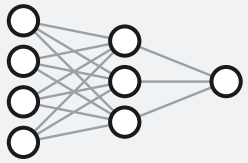
Saitama University



Slides (PDF)

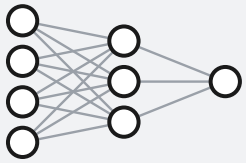
Compressing many-body physics: a parallel route

Wavefunction
$\Psi(\sigma_1, ..., \sigma_N)$



ground states

Free energy
$F = -T \ln Z$



finite temperature

Green's function
$G(\boldsymbol{k}, i\omega_n)$



dynamics, response

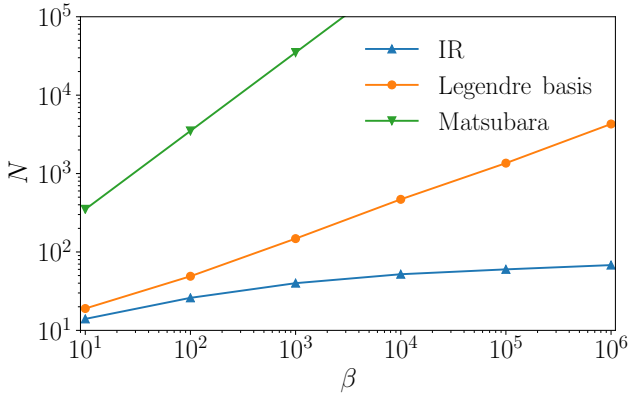
neural networks

low-rank structure

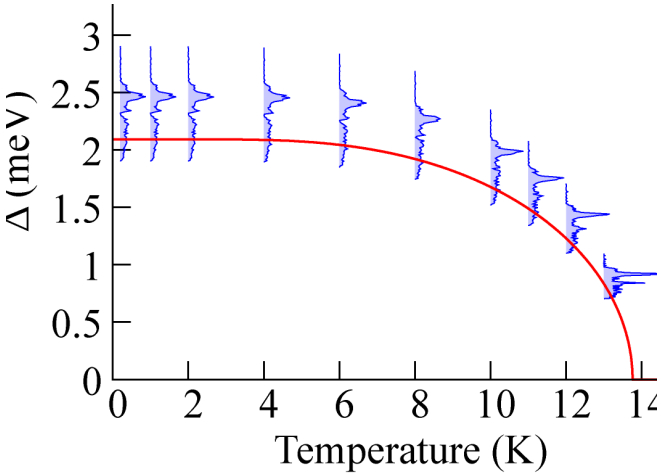
Our route: compress **Green's functions** by low-rank structure

Carleo, Troyer, Science 355, 602 (2017). Nomura, ..., Imada, PRB 96, 205152 (2017). Xie, ..., L. Wang, PRL 131, 126501 (2023). IR: Shinaoka et al., PRB 96, 035147 (2017). QTT: Shinaoka et al., PRX 13, 021015 (2023). TCI: Ritter et al., PRL 132, 056501 (2024); Núñez Fernández et al., SciPost Phys. 18, 104 (2025).

IR basis: Matsubara Green's functions are low-rank



Basis size: IR grows only as $\log \beta$



Nb, *ab initio* (EPW): gap down to $O(1)$ **K**

450 citations

sparse-ir Python

SparseIR.jl Julia

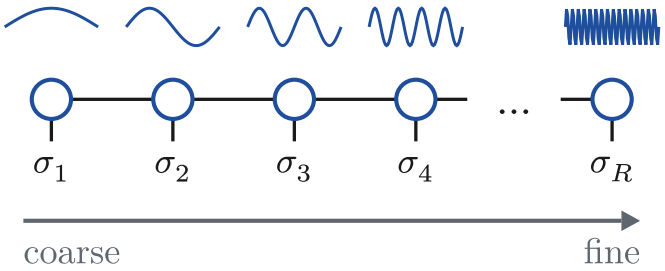
sparse-ir-fortran

used in **EPW** (electron-phonon)

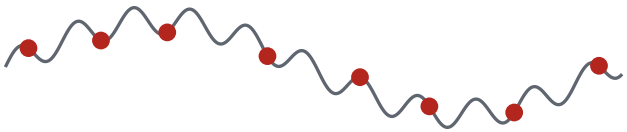
IR: intermediate representation. Shinaoka et al., PRB 96, 035147 (2017).
sparse-ir: SoftwareX 21, 101266 (2023). Nb: Mori et al., PRB 110, 064505 (2024).

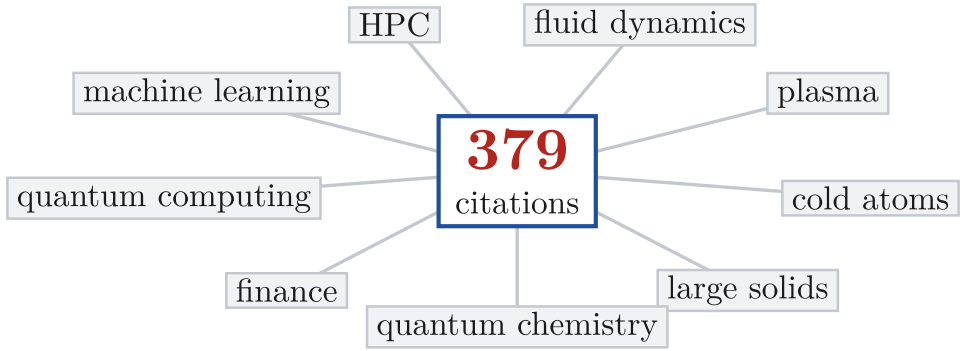
QTT/TCI: compression by scale separation

Quantics tensor train (QTT):
one tensor per length scale



Tensor cross interpolation (TCI):
learned from a few samples



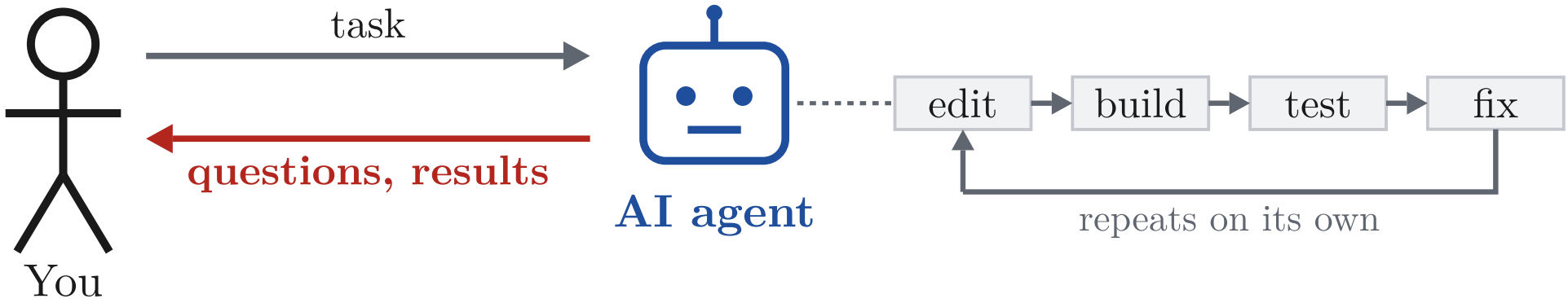


libraries in **C++** / **Python** / **Julia**

So why rebuild it all in Rust with AI agents, now?

Shinaoka et al., PRX 13, 021015 (2023); Ritter et al., PRL 132, 056501 (2024).
Citations: Google Scholar (2026-11-04).

What is an AI agent?



Example: **1 agent, 4 hours**: 8,200 lines of Rust
porting the post-processing of EPW (electron-phonon code)
matches EPW output, e.g. MgB_2 $T_c = 26.414$ K (unpublished)

Two questions

Why Rust?

Why not C++ or Julia?

How to grow and maintain it with AI?

A huge code base



A fragmented ecosystem

Machine learning	QMC	Tensor networks	Ab initio
Python	**C++**	**Julia / Python**	**Fortran / C++**
PyTorch JAX	ALPS TRIQS QMCPACK	ITensors.jl TeNPy quimb	VASP Quantum ESPRESSO EPW

Each field has its own language and build system

How can we integrate our compression into high-performance computing in various fields?



Why Rust?

	C++ Fortran	Julia	Rust
No segfault after hours of running	✗	✓	✓
No silently wrong results (aliasing)	✗	✗	✓
Parallel without garbage collector	✓	✗	✓
One package manager and build tool	✗	✓	✓
Callable from Fortran, Python, Julia	✓	✗	✓

One Rust core, callable from every field's code



Rust is also used in the Linux kernel (since 6.1, 2022).

But a steep learning curve

Julia

```julia
function normalize!(x)
  s = sum(abs2, x)
  x ./= sqrt(s)
end
```

Rust

```rust
fn normalize<T>(x: &mut [T])
where T: Float + DivAssign {
    let n = x.iter()
        .fold(T::zero(),
              |a, &v| a + v * v)
        .sqrt();
    for v in x.iter_mut() {
        *v /= n;
    }
}
```

Right: hard for most physicists

With AI agents, the learning curve and readability no longer matter

Industry has noticed too

David Heinemeier Hansson

creator of Ruby on Rails, Rails World 2026 keynote

"Rust to me is like pouring acid in my eyes"

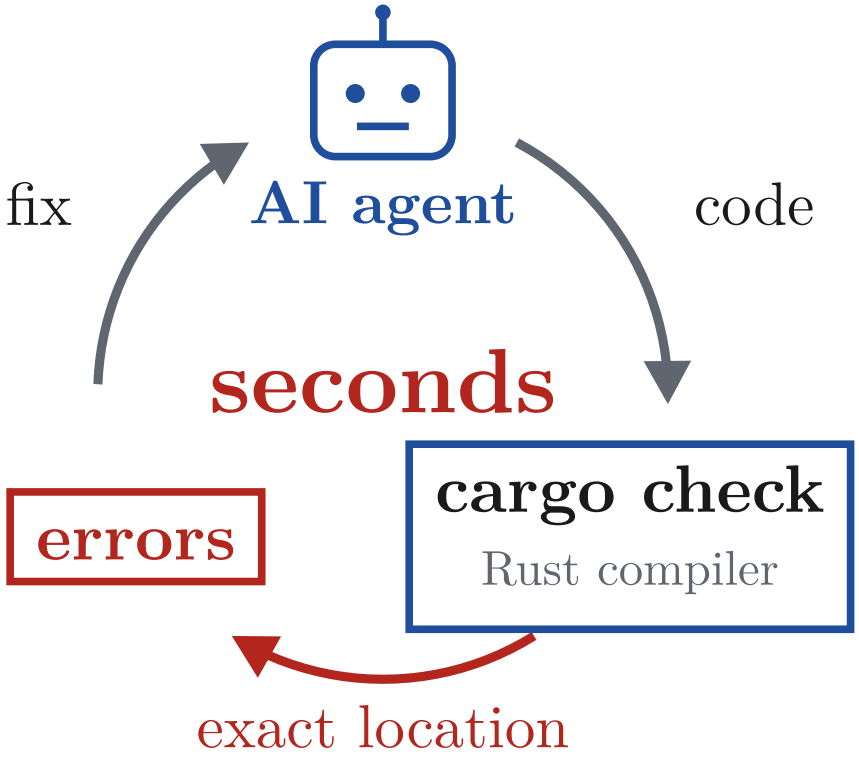
"I love Rust! Rust is amazing
if you never, ever, ever have to look at it yourself."

"Agents like Rust."



Video: youtu.be/vDjW_dRyKXY?t=1725 (28:45)

In the AI era, strong checks matter



Caught before running:

- ✗ type mismatch (f32 vs f64)
- ✗ data race between threads
- ✗ use of freed memory
- ✗ unhandled case in `match`

The compiler checks every line the AI writes, **before it runs**

Our software stack

tensor4all-rs **350k**

tensor networks: TCI, QTT

sparse-ir-rs **66k**

compressed Green's functions

tenferro-rs **360k lines**

tensors, linear algebra, autodiff



tensor4all.org

★ Please star us on GitHub!

360k lines since 2026-02-13

tenferro-rs

tensors, einsum, FFT, linear algebra,
autodiff (eager and traced)



einsum cpueinsum-rs	**contraction** tcontract-rs	**decompositions** tlinalg-rs
strided arrays strided-rs	**dense kernels** t4a-faer (fork)	**threads** threadteam-rs

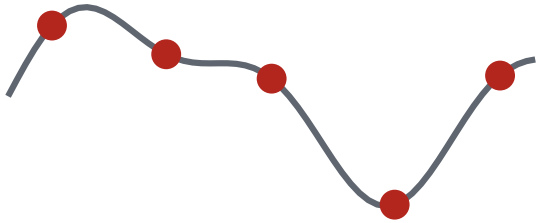
vs PyTorch on CPU and NVIDIA A100: tensor4all.org/tenferro-benchmark
github.com/tensor4all/tenferro-rs

Tensor trains



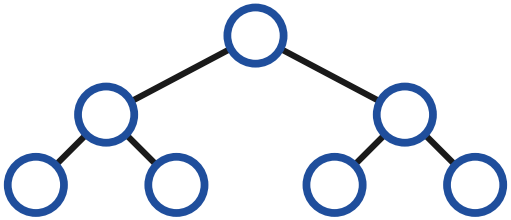
QTT: 2^R grid points

TCI



built from a few samples

Tree networks

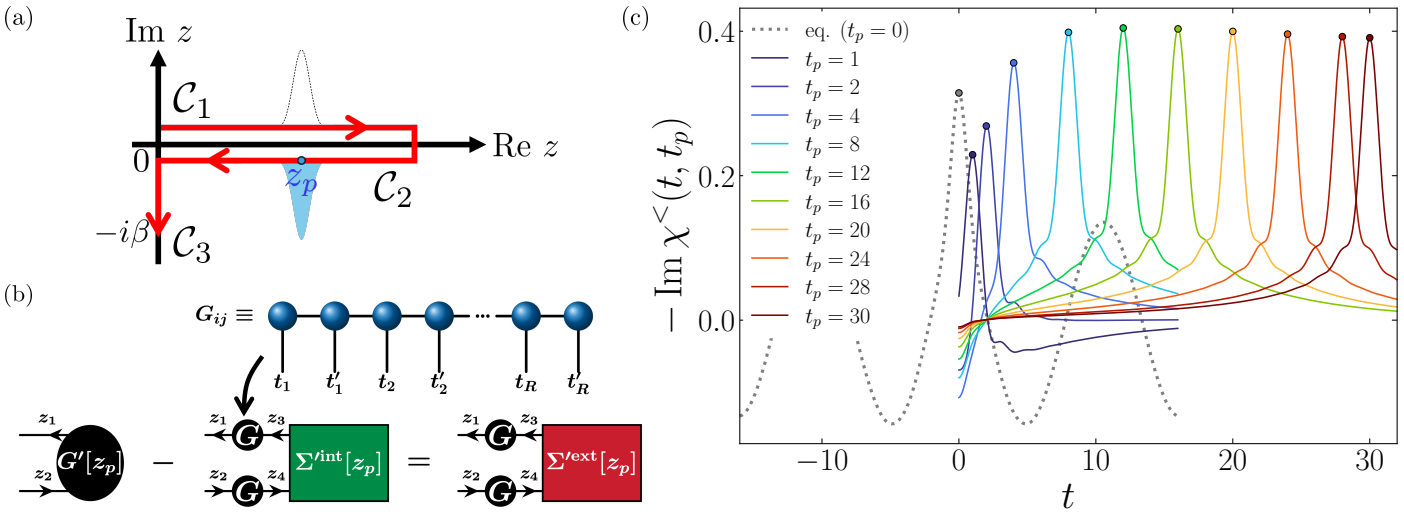


any topology

Built on tenferro-rs, with a C API and
an ITensor-compatible Julia interface

TCI: tensor cross interpolation. QTT: quantics tensor train.
github.com/tensor4all/tensor4all-rs

First result on this base: nonequilibrium in QTT





Ken Inayoshi

K. Inayoshi, H. Shinaoka, Y. Murakami, arXiv:2607.11055 (2026)

- Quench in the Hubbard model
- Memory: dense grid 0.29 GB → QTT ∼ 1e-3 GB
- First result 2 months after he started Rust

20 contributors, 7 countries

1,362
PRs merged
in the last 30 days

Japan 6
H. Shinaoka, S. Terasaki, R. Watanabe
K. Inayoshi, R. Sakurai, H. Ishida

Austria 3
S. Badr, S. Dirnböck, N. Ritz

Germany 2
L. Cheng
M. Frankenbach

USA 2
L. Devos
M. Ritter

France 3
S. Becker, T. Kloss
Y. Núñez Fernández

Denmark 1
M. Mikkelsen

China 3
J. Liu, Y. Zhao, X. Pan

Humans in 2026, tensor4all repositories (Rust and Julia, no forks); all use AI agents.

Two questions

Why Rust? ✓

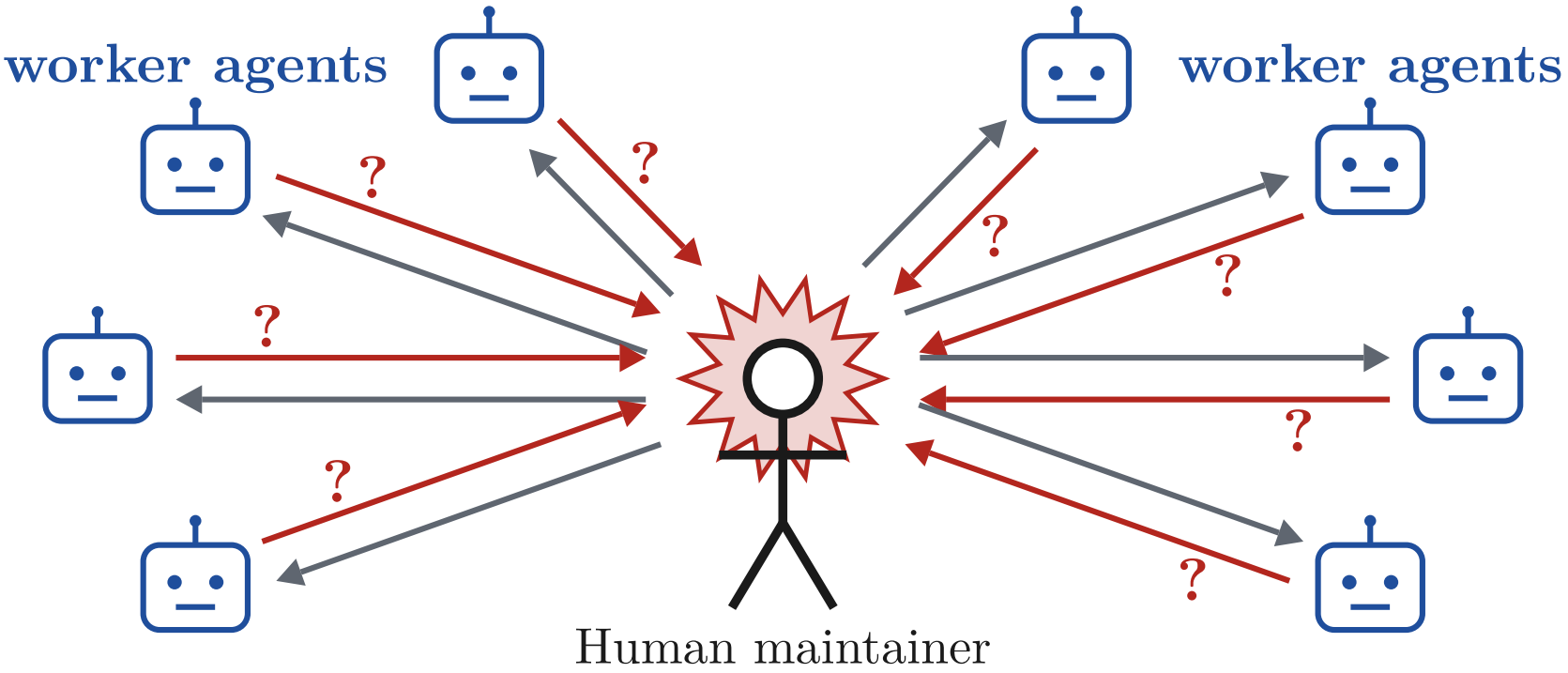
Why not C++ or Julia?

How to grow and maintain it with AI?

A huge code base



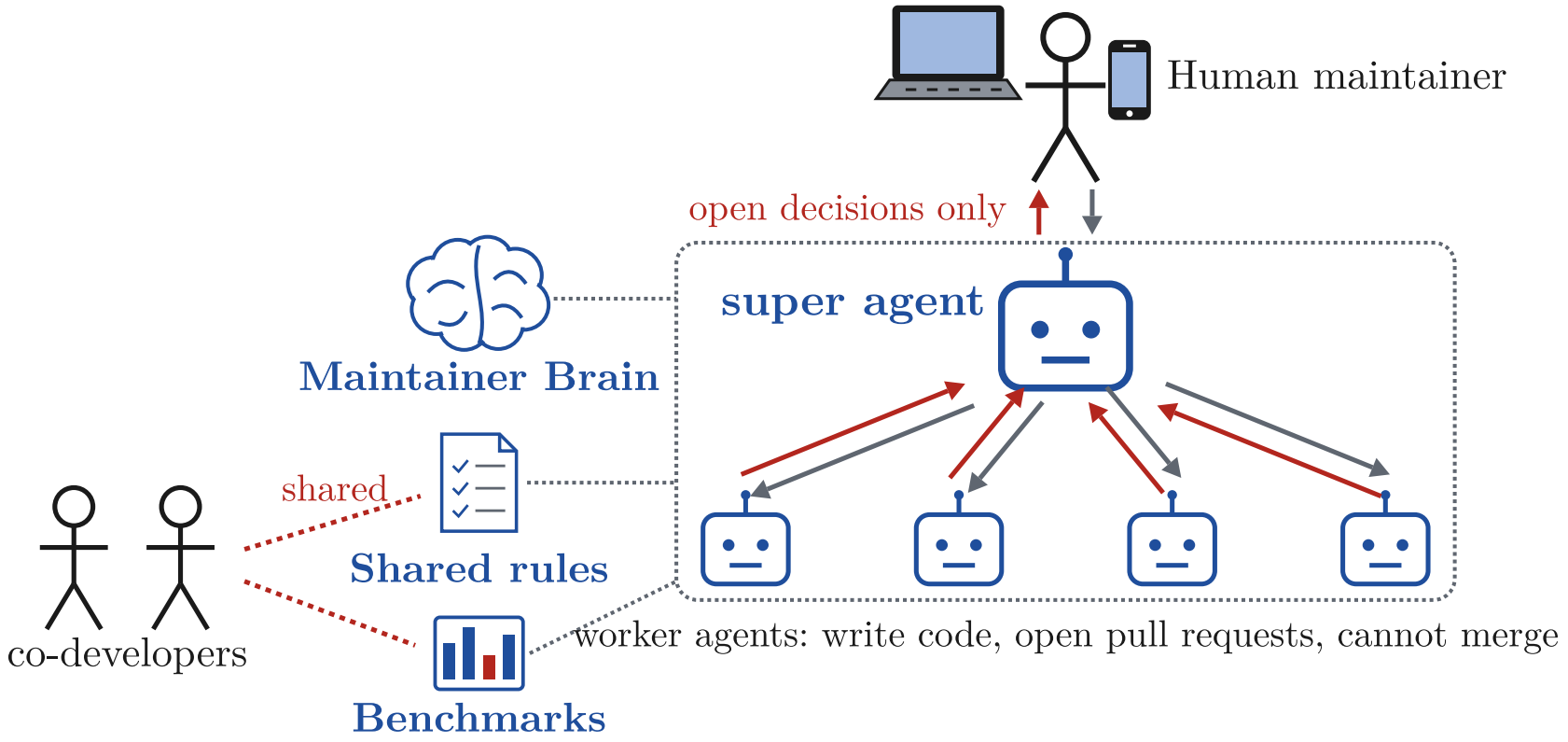
The human is the bottleneck



Human attention is a scarce resource

spent on answering, deciding, and bookkeeping

A super agent handles a swarm of worker agents



tensor4all.org/blog/how-we-check-agent-written-code

DeepSeek's super-agent prototype (2026-10-31) follows the same principle

What the super agent relies on



Maintainer Brain

the maintainer's judgment, distilled from past decisions

310 rules, 1,536 decisions

maintainer only

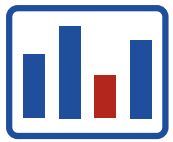


Shared rules

expert tricks and project conventions

19 rule files, 4 skills

shared with co-developers



Benchmarks

public sites, run regularly

tenferro, faer, tlinalg, ...

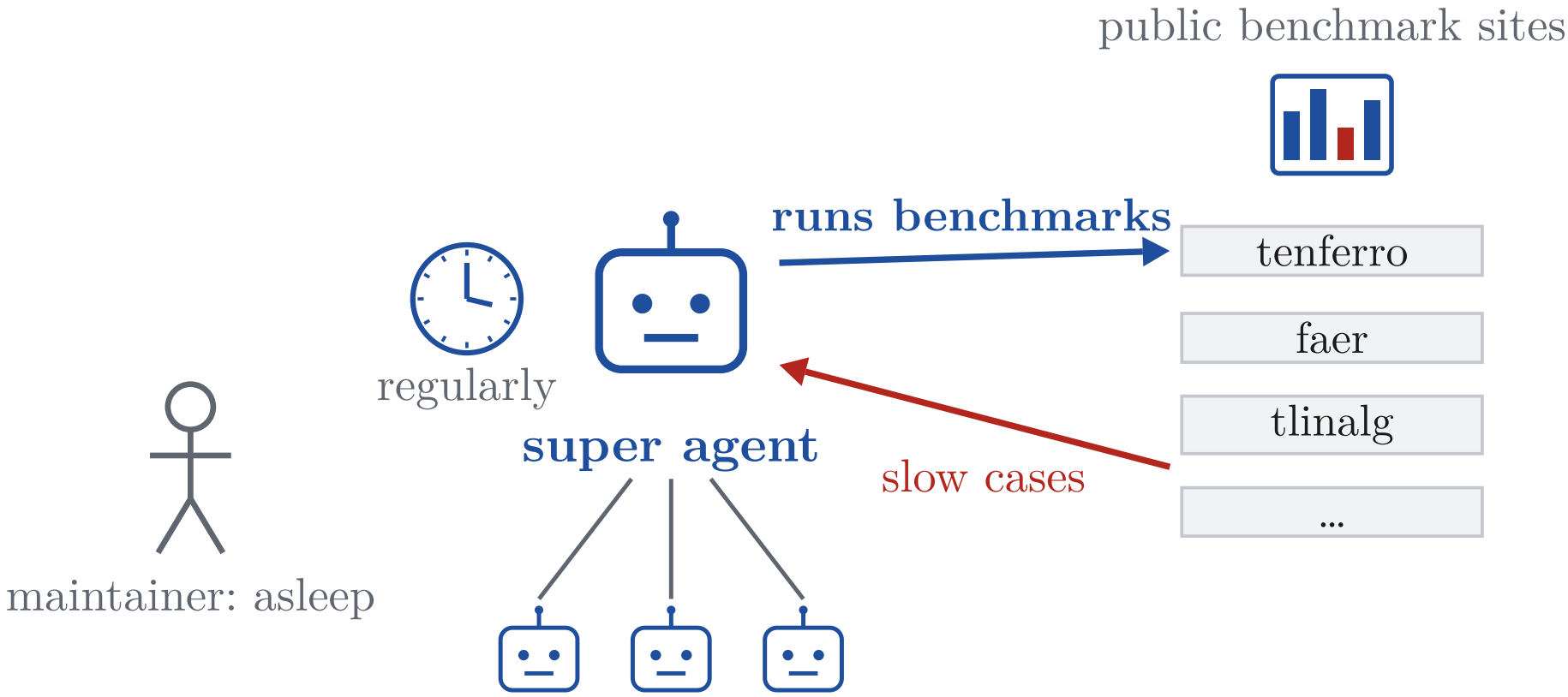
shared with co-developers

Growing: all three are updated adaptively after each mistake



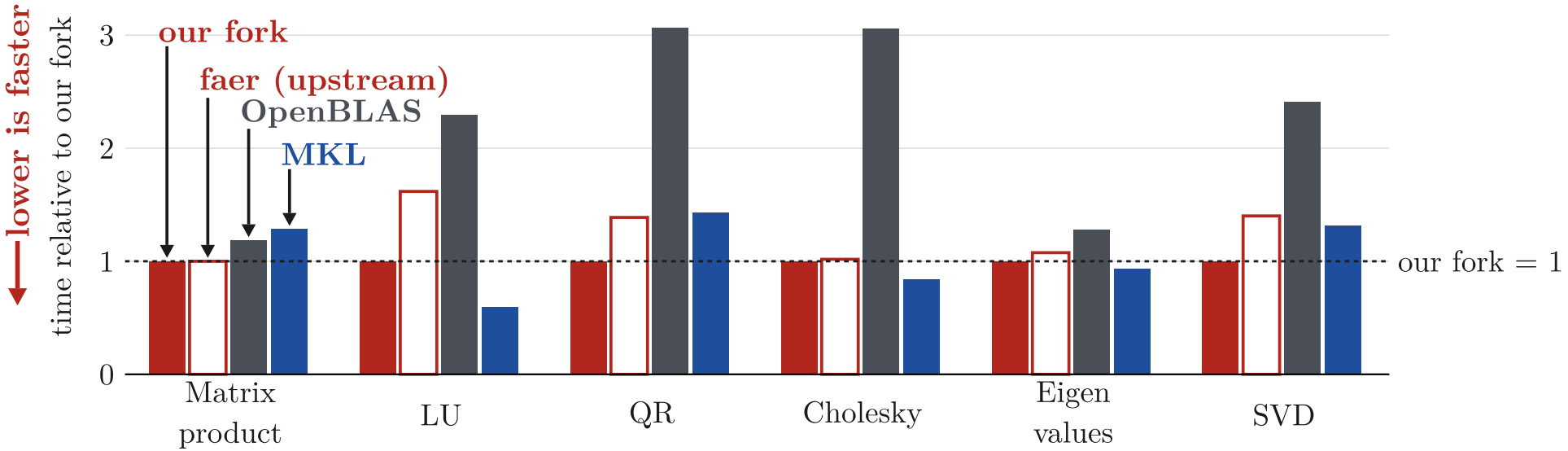
Brain: decisions from 2026-05-20 to 10-28. Idea: github.com/QuantumBFS/sci-brain

Automatic optimization with public benchmarks



Sites: tensor4all.org/tenferro-benchmark, /faer-benchmark, /tlinalg-benchmark, ...

Case study: our fork of faer, a Rust linear algebra library



Faster than upstream faer and OpenBLAS. MKL: mixed.

Next: C API for C/C++, Python, Julia, Fortran

Tuned by the benchmark loop. 1024 × 1024, f64, 8 threads, Ryzen AI 9 HX 470.
Matrix product ties upstream. tensor4all.org/faer-benchmark. Upstream: bug reports only.

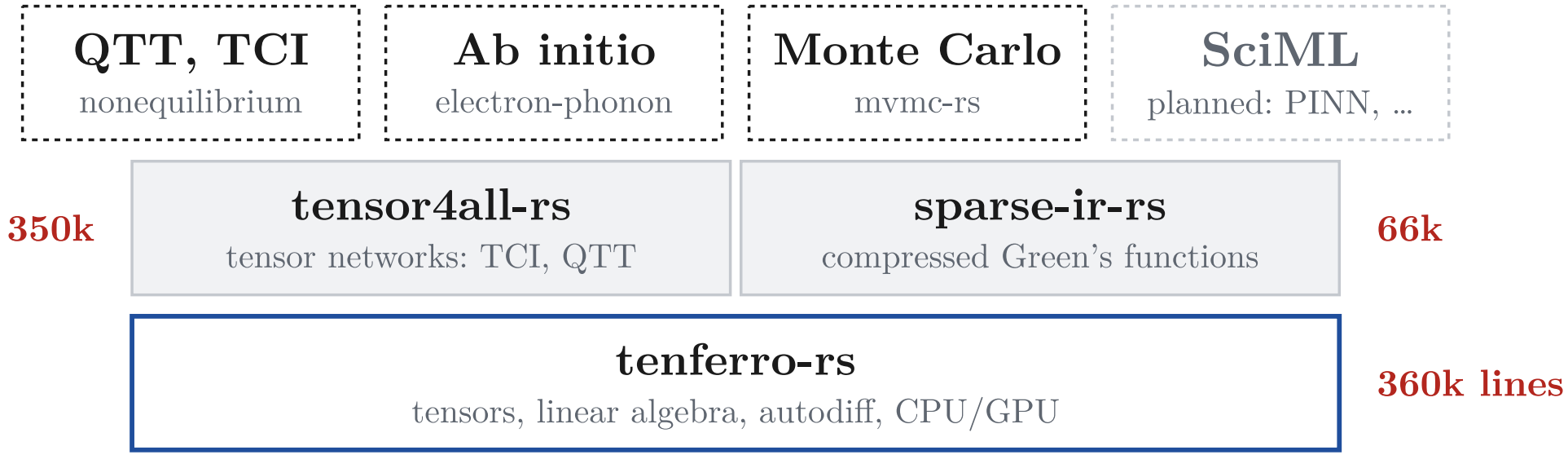
Correctness: tests ported by AI find bugs



Example: SVD workspace fix for f32 (tensor4all/faer#73)

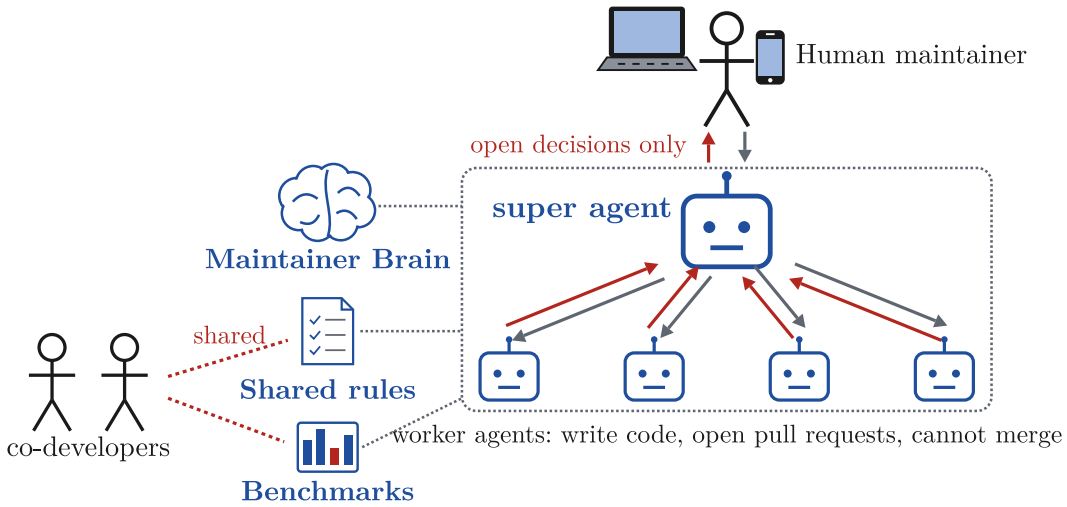
LAPACK test port and fixes: **M. Mikkelsen**. Further fixes: **S. Terasaki**.

New fields and people are joining



Nonequilibrium: **K. Inayoshi** (arXiv:2607.11055).
mvmc-rs (outside tensor4all): **T. Misawa**, **S. Terasaki**. SciML: tenferro-rs#2419.

Summary



Slides (PDF)

tensor4all.org

Why Rust? compiler checks every line; one core for every field

How to maintain? super agent with Brain, shared rules, benchmarks

A new, growing Rust ecosystem for computational physics

Join us!

Inayoshi et al., arXiv:2607.11055
github.com/QuantumBFS/sci-brain
codeberg.org/sarah-quinones/faer/issues/340
github.com/tensor4all/tensor4all-agent-rules
tensor4all.org/blog (2026-11-04)
github.com/deepseek-ai/dsh-superagent-prototype
github.com/AtelierArith/mvmc-rs

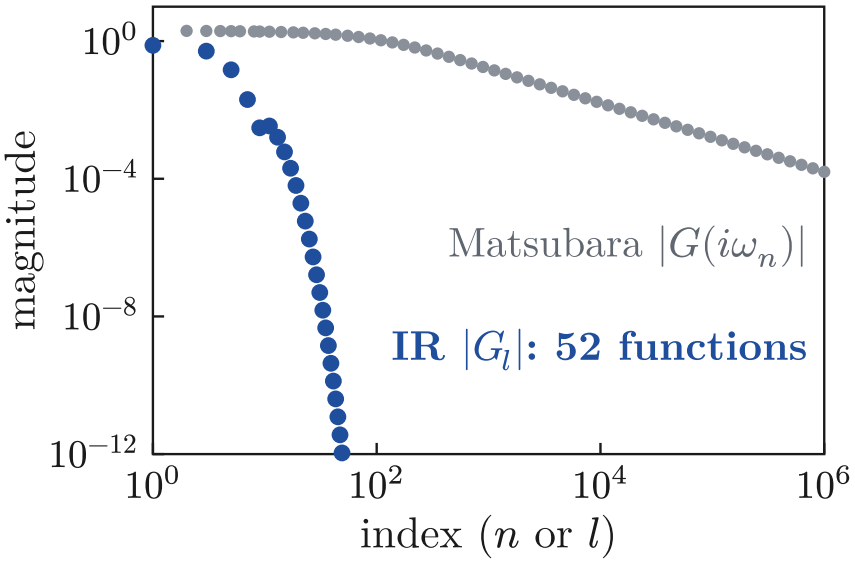
Cost per month

Model, role	Service	Cost
Collaborators		
Claude	Claude Code Team premium seat	**$20** (list $100)
Me		
Claude worker for hard problems	Claude Code Team premium seat	$20
GPT6.1-sol reviewer	ChatGPT / Codex	$100
DeepSeek V4.1 Flash	Ollama Cloud	$100
workers, super agent	OpenCode Go	$40

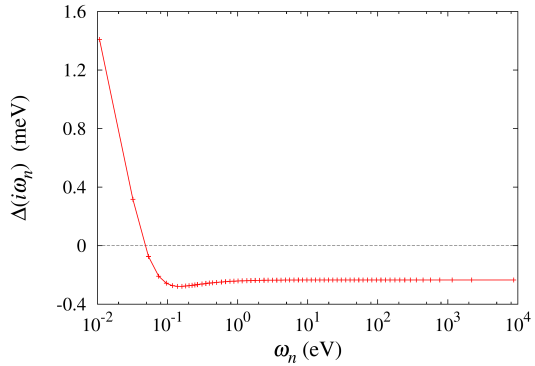


Most coding tasks: **DeepSeek is enough**

Backup: the IR basis



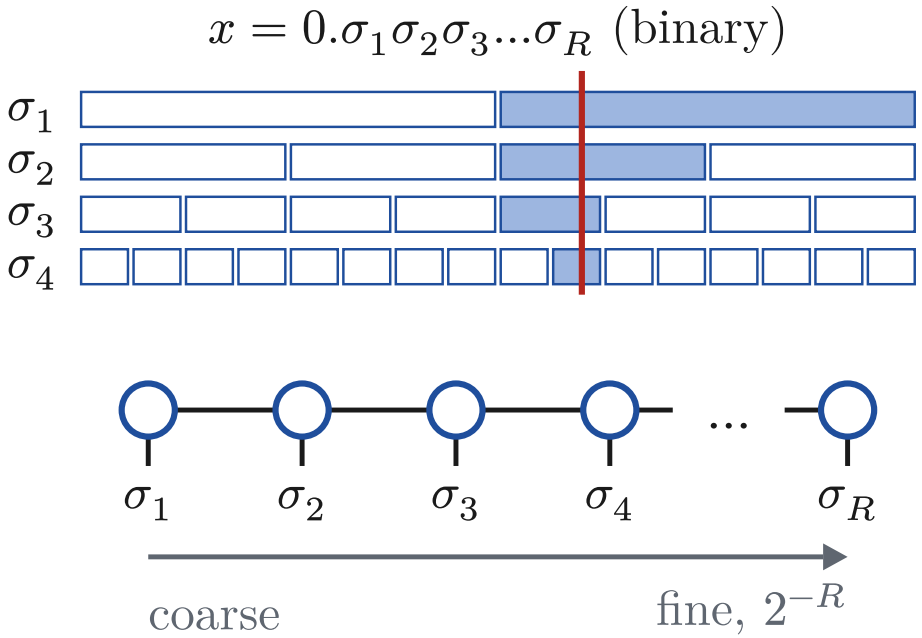
IR: **52 numbers** reach 10^{-12}

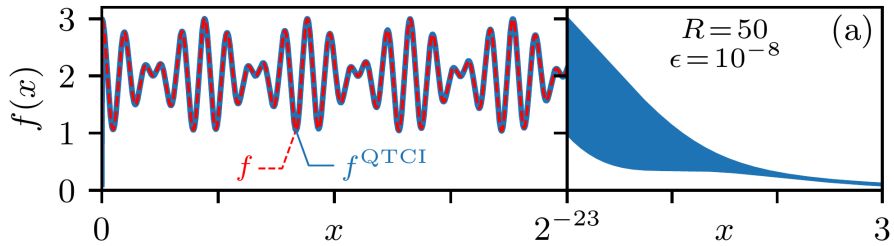


Eliashberg, Nb: **138 points**
instead of dense Matsubara grids

IR (intermediate representation): Shinaoka et al., PRB 96, 035147 (2017).
Sparse sampling: Li et al., PRB 101, 035144 (2020).
Right: Wang, ..., Arita, PRB 102, 134503 (2020).

Backup: QTT, one tensor per scale





2^{50} points, scales 2^{-23} to 3

bond dimension 15

Right: Ritter et al., PRL 132, 056501 (2024).

Backup: compile-time check of a data race

C++ / OpenMP

```
#pragma omp parallel for
for (int i = 0; i < n; ++i)
  total += x[i];
```

Compiles; wrong sum at run time

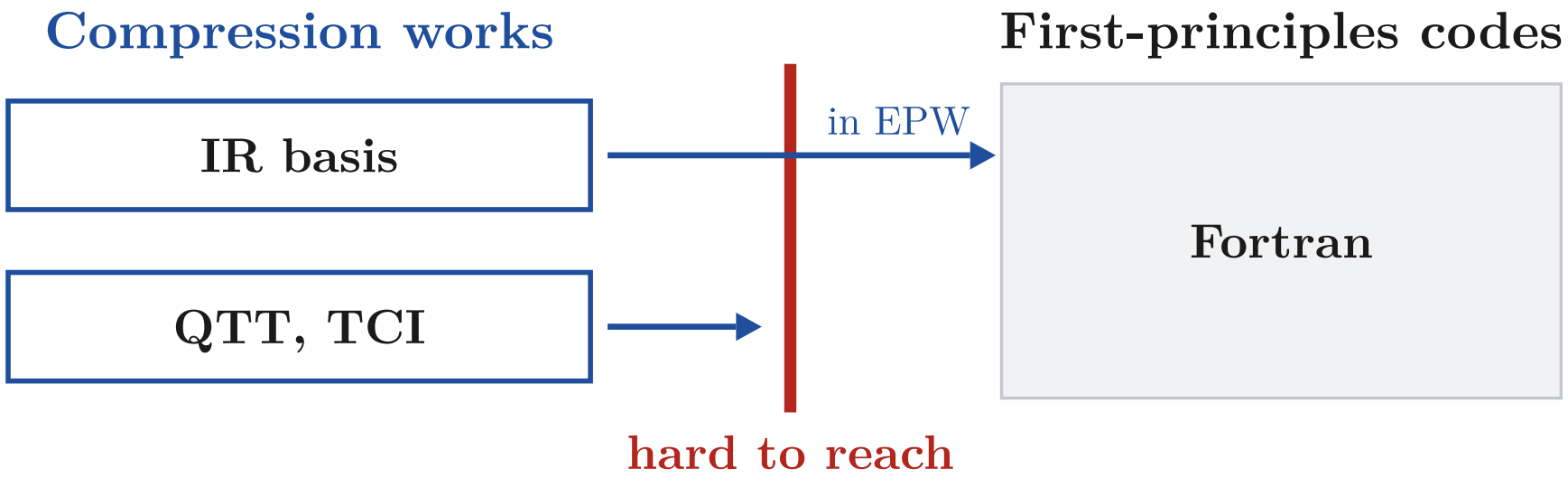
Rust / rayon

```
x.par_iter()
  .for_each(|xi| total += xi);
```

```
error[E0596]: cannot borrow
`total` as mutable
```



Backup: compression is hard to bring into ab initio codes



Our own libraries: four languages, each maintained separately

Python | Julia | C++ | Fortran

So we rebuilt one Rust base with AI agents

QTT: quantics tensor train. TCI: tensor cross interpolation.
EPW: electron-phonon code. IR in EPW: Mori et al., PRB 110, 064505 (2024).